

Service Language Selection & Architecture Flow

Phase 1 baseline · 2026-07-11 · Companion to *ARCHITECTURE.md* · Status: Approved

1. Governing Principle: Two Languages, Not Five

Every language added to the stack multiplies CI pipelines, observability tooling, dependency scanning, hiring surface, and on-call knowledge. Themistra standardises on exactly **two application languages**:

- **Java 21** for every backend service — the team's core competency, and the JVM's maturity matches a product whose entire value is *correctness of record*.
- **TypeScript** for the frontend, for infrastructure as code (AWS CDK), and for narrowly-scoped chain-adapter sidecars where the web3 ecosystem demands it.

A third language is only admitted through an ADR demonstrating that both existing languages materially fail the use case.

2. Language per Service — Summary

COMPONENT	LANGUAGE / STACK	PRIMARY REASON
Edge / API Gateway	Config, not code ingress-nginx + ALB/WAF	Lean start: CloudFront → WAF → ALB → ingress-nginx (EKS) handles routing, rate limits and TLS via annotations. JWT validation lives in each service (OAuth2 resource server) — zero trust, required regardless. Graduates to a real gateway on merchant API keys / partner API / BFF aggregation.
Auth Service	Java 21 Spring Boot + Spring Security + Spring Authorization Server	The most battle-tested OIDC/OAuth2 stack in existence. Auth is where novelty is a liability — we want boring, audited, CVE-responsive maturity.
Payment & Verification Service	Java 21 Spring Boot + JPA/Flyway	Domain-heavy and transactional: outbox writes, the verification state machine, hash-chain ledger, and money arithmetic (BigDecimal). Strong typing makes illegal state transitions unrepresentable.
Crypto Service (core)	Java 21 Spring Boot, virtual threads, web3j / Tron gRPC	Quorum comparison, finality policy, reorg handling and KMS attestation are correctness-critical and belong beside the security core. Virtual threads make thousands of concurrent watcher connections cheap.
Crypto Service (chain sidecars)	TypeScript Node, viem / @solana/web3.js	Web3 innovation lands in TypeScript first. Where Java tooling is thin (Solana; some provider SDKs), a thin TS adapter translates chain APIs into our Kafka/gRPC contract — the contract is the boundary, not the language.
Notification Service	Java 21 Spring Boot	A Kafka consumer with templates and delivery logging — nothing here demands another runtime, so operational uniformity wins.
Frontend	TypeScript React (mobile-first PWA)	Never plain JavaScript: payment amounts, chain identifiers and wallet addresses deserve compile-time types, generated from the backend's OpenAPI spec.
Infrastructure as Code	TypeScript AWS CDK	We are AWS-domiciled, so Terraform's portability buys nothing. CDK keeps the two-language rule intact, type-checks infrastructure at compile time, and lets us write reusable constructs (one ThemistraService construct stamps out identical least-privilege scaffolding per service). No console-created resources.
Managed substitutions	AWS-managed	MSK (Kafka), RDS (Postgres), KMS, S3 Object Lock — buy the undifferentiated heavy lifting.

3. Rationale in Depth

3.1 Why Java everywhere on the backend — including the crypto service

The tempting alternative is Go for the watcher layer (classic concurrency story) or full TypeScript for the crypto service (richest web3 libraries). We reject both as *primary* choices:

- **Java 21 virtual threads removed Go's main advantage.** Holding tens of thousands of provider websocket subscriptions no longer needs a special runtime.
- **The crypto service's hard parts are not chain I/O.** They are quorum arbitration, per-chain finality policy, reorg-safe state, and the KMS signing path — audit-critical logic that benefits from the same typing, testing and review culture as the payment ledger it protects.
- **web3j and Tron's gRPC interface fully cover our launch chains** (Ethereum, Tron). We pay the sidecar cost only when a chain actually demands it.

3.2 The TypeScript sidecar escape hatch

When a chain's Java tooling is inadequate, we add a minimal Node sidecar that speaks the chain's native SDK on one side and our internal adapter contract on the other. Rules:

- Sidecars **observe and translate only** — no quorum decisions, no signing, no business state.
- Each sidecar is one chain, one process, independently deployable and disposable.
- The Java core treats sidecar output as just another provider answer, subject to the same 2-of-3 quorum.

Security consequence: everything that can cause a false attestation lives in one language, one review pipeline, one dependency-scanning regime. Sidecars can fail or lie without compromising an attestation, because they never hold quorum authority or key access.

3.3 Where we deliberately buy instead of build

Kafka (MSK), Postgres (RDS), key custody (KMS), immutable storage (S3 Object Lock) and edge protection (WAF/CloudFront) are managed services. Engineering hours go into the two things competitors cannot copy from a vendor: the quorum verification engine and the tamper-evident record.

4. System Topology

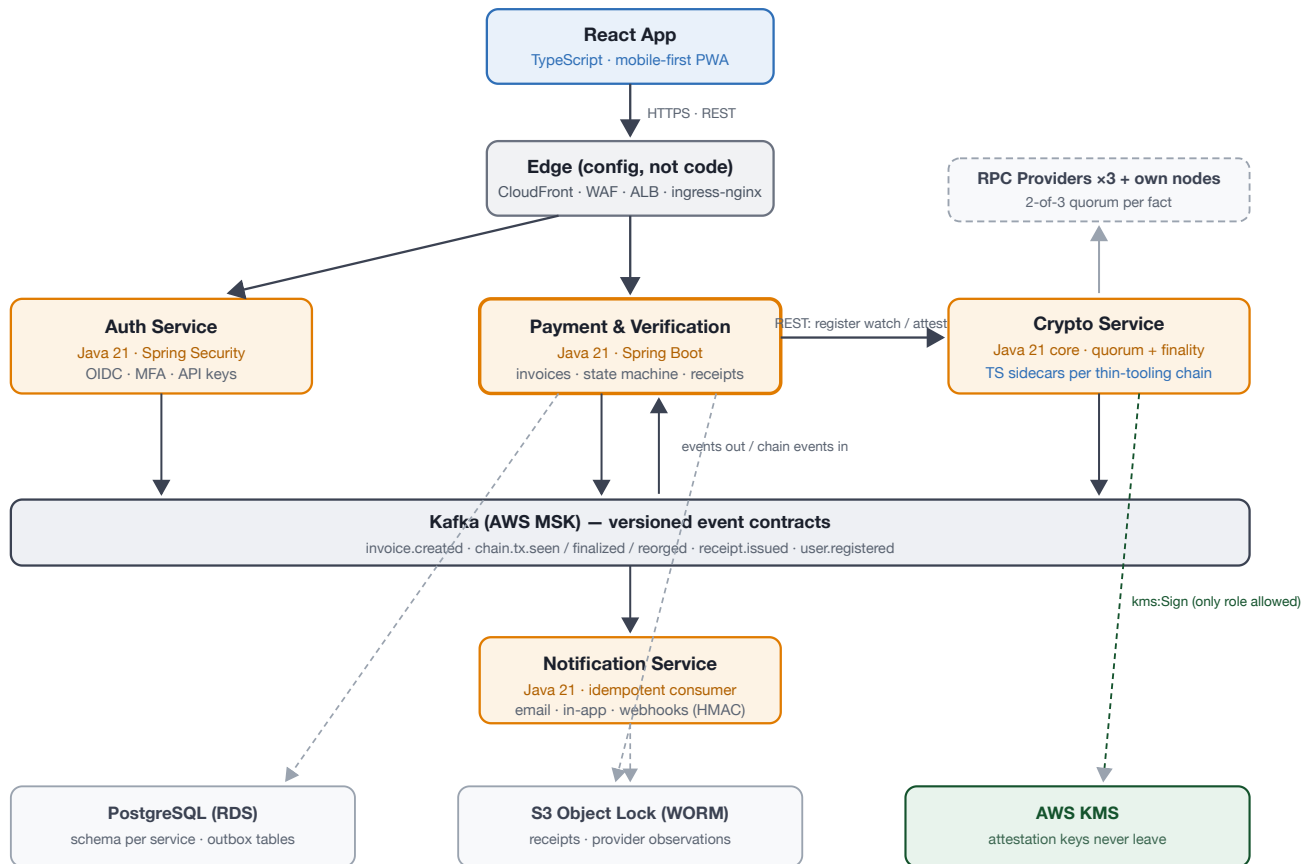


Fig. 1 — Phase 1 topology with language assignment. Orange = Java 21, blue = TypeScript, gray = managed AWS. Dashed lines are data-plane; every service also owns its own Postgres schema (arrows abridged for clarity).

5. Core Flow — Verified Payment End to End

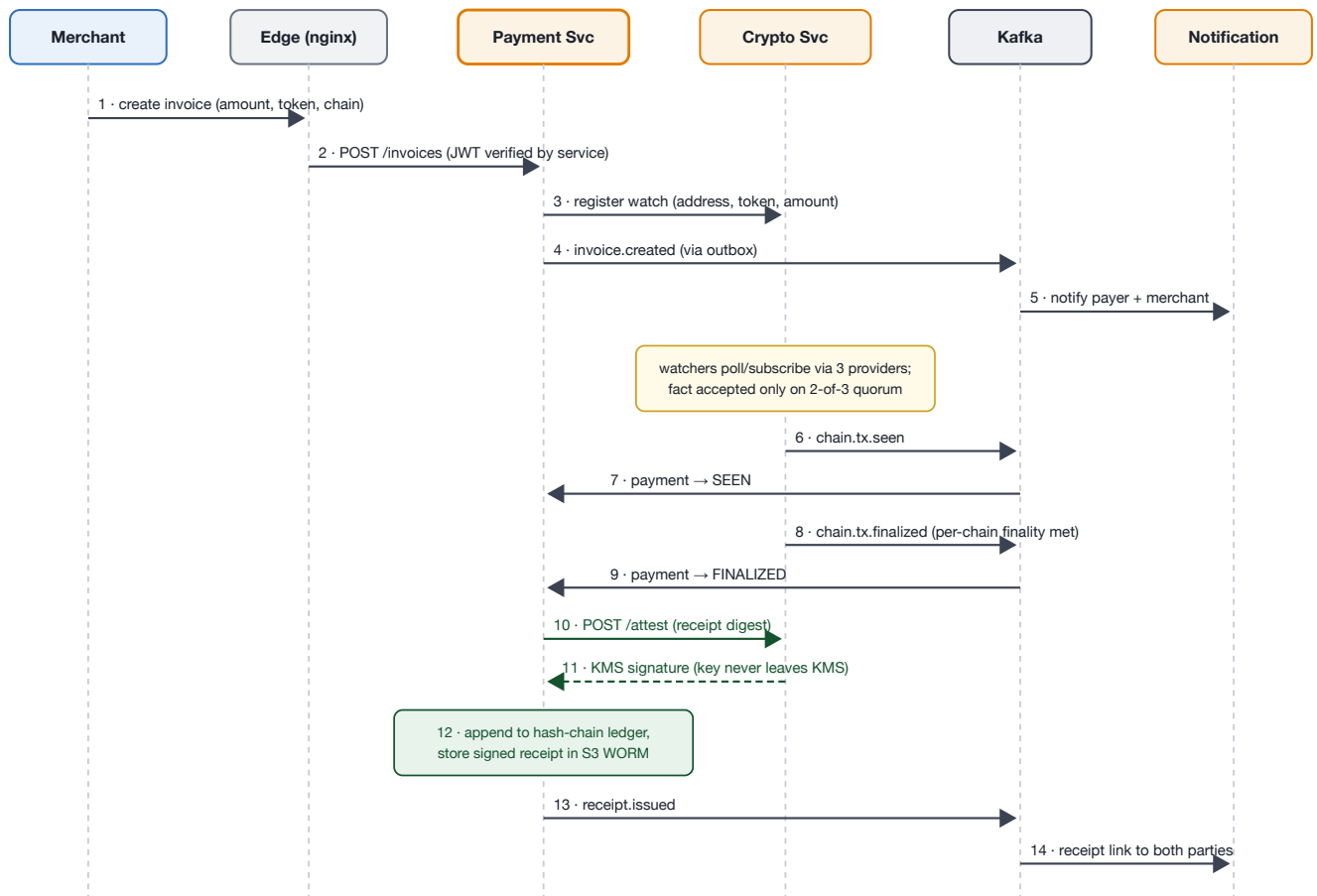


Fig. 2 — Verified-payment sequence. Green steps (10–12) are the attestation path: signing happens only after quorum-agreed finality, and the record becomes tamper-evident before anyone is notified. A reorg between steps 6 and 8 emits `chain.tx.reorged` and walks the state machine backward — no receipt exists yet, so nothing needs revoking.

6. Decision Record

DECISION	CONSEQUENCE WE ACCEPT
Java 21 for all backend services	Slightly slower access to newest web3 SDKs; mitigated by the TS sidecar pattern.
TypeScript sidecars are translation-only	Some duplicated chain models across the boundary; accepted to keep quorum/signing authority in one audited codebase.
No Go / Python / Kotlin in Phase 1	Python enters in Phase 2 with the Intelligence Service (ML ecosystem) via a new ADR — the event contracts are already language-neutral.
AWS CDK (TypeScript) over Terraform	CloudFormation semantics: slower deploys, occasionally painful rollbacks, weaker drift detection than <code>terraform plan</code> , and deeper AWS coupling — all accepted since we are AWS-domiciled and gain a type-checked IaC in a language we already run.
ingress-nginx instead of a gateway service	No edge-level JWT rejection or response aggregation for now; every service validates tokens itself (needed anyway for zero trust). Revisit on merchant API keys, partner API plane, or BFF aggregation needs.
Managed AWS for Kafka, Postgres, keys, storage	Vendor coupling; accepted — our differentiation is the verification engine, not infrastructure operation.